

BÁO CÁO THỰC NGHIỆM: AI TRÒ CHƠI CỜ CARO

Họ và tên: Nguyễn Phúc Gia Bảo

Mã sinh viên: 24022265

1. Mô tả bài toán và Biểu diễn trạng thái

- Mô tả bài toán: Xây dựng chương trình AI cho trò chơi cờ Caro (Gomoku phiên bản thu gọn). Hai người chơi (Người - X và Máy - O) luân phiên đánh dấu vào các ô trống trên bàn cờ.
- Luật chơi và Điều kiện thắng: Bàn cờ sử dụng có kích thước 9x9. Trò chơi kết thúc khi một người chơi tạo được chuỗi 4 quân cờ liên tiếp theo chiều ngang, dọc hoặc chéo. Không áp dụng luật chặn hai đầu. Nếu bàn cờ kín ô mà không có người chiến thắng, kết quả là hòa.
- Biểu diễn trạng thái: Bàn cờ được biểu diễn bằng một ma trận 2 chiều thông qua thư viện numpy để tối ưu tốc độ xử lý. Các giá trị trong ma trận được quy ước: 0 (Ô trống), 1 (Người chơi X), và -1 (AI O).
- Sinh nước đi hợp lệ: Thuật toán duyệt qua toàn bộ ma trận và trả về danh sách tọa độ \$(row, col)\$ của tất cả các ô có giá trị bằng 0.

2. Thuật toán và Hàm đánh giá

- Thuật toán Minimax & Alpha-Beta: Chương trình sử dụng thuật toán Minimax để xây dựng cây trò chơi và chọn nước đi tối ưu dựa trên giá định đối thủ cũng chơi hoàn hảo. Nhằm giải quyết độ phức tạp không gian và thời gian \$O(bd)^d\$ (với \$b\$ là hệ số phân nhánh, \$d\$ là độ sâu), thuật toán Alpha-Beta Pruning được tích hợp để cắt bỏ các nhánh không ảnh hưởng đến kết quả cuối cùng.
- Hàm đánh giá (Heuristic Evaluation): Do không thể duyệt đến trạng thái kết thúc của ván cờ, một hàm đánh giá được sử dụng tại các node lá (khi đạt giới hạn độ sâu). Ý tưởng cốt lõi là trượt một "cửa sổ" kích thước 4 ô dọc theo các hàng, cột và đường chéo.
 - Thuật toán đếm số lượng quân của AI và số lượng ô trống trong cửa sổ để cộng điểm thưởng (tấn công).
 - Đồng thời, đếm số lượng quân của đối thủ để trừ điểm (phòng thủ). Trọng số phòng thủ được đặt cao hơn tấn công (ví dụ: phát hiện đối thủ có 3 quân mở sẽ bị trừ \$100,000\$ điểm) để ép AI ưu tiên chặn đường thắng của người chơi.

3. Thiết kế thực nghiệm

Để đánh giá hiệu quả của thuật toán, chương trình thiết lập các trạng thái kiểm thử đặc trưng:

1. Trạng thái đầu ván: Bàn cờ trống, chỉ có 1 nước đi duy nhất ở trung tâm.

2. AI cần chặn người chơi: Người chơi X đã có 3 quân, AI buộc phải tìm ra nước đi chặn để không bị thua.
3. AI tấn công để thắng: AI đã có 3 quân, cần tìm nước đi quyết định để kết thúc ván đấu.
4. Trạng thái giữa ván: Thế trận giằng co, đan xen giữa các nước tấn công và phòng thủ.
5. Trạng thái phân tán: Nhiều quân cờ nằm rải rác, hệ số phân nhánh lớn.

4. Kết quả và Đánh giá thực nghiệm

Dưới đây là kết quả chạy benchmark cho hai thuật toán trên cùng cấu hình hàm đánh giá.

- Đánh giá sự đồng nhất về nước đi

Thực nghiệm cho thấy ở tất cả các Test case, Alpha-Beta luôn trả về cùng một tọa độ nước đi với Minimax (Ví dụ: tại Test 2, cả hai đều chọn tọa độ (0, 0) để chặn đối thủ). Điều này chứng minh thuật toán Alpha-Beta đã được cài đặt chính xác, chỉ tối ưu quá trình duyệt (cắt nhánh) chứ không làm sai lệch kết quả tối ưu của Minimax.

- Phân tích hiệu năng cắt nhánh và thời gian chạy

Sự vượt trội của Alpha-Beta được thể hiện rõ nhất khi tăng độ sâu tìm kiếm (Depth):

- Tại Độ sâu (Depth) = 2:
 - Ở trạng thái giữa ván (Test 4), Minimax phải xét 5,477 trạng thái (tốn ~1.61s).
 - Alpha-Beta chỉ cần xét 1,975 trạng thái (tốn ~0.57s), giảm được khoảng 63.94% không gian tìm kiếm.

TEST CASE: 2. AI cần chặn người chơi (Độ sâu: 2)			
=====			
Thuật toán	Nước đi	Số node đã xét	Thời gian (s)

Minimax	(0, 0)	6085	1.6671
Alpha-Beta	(0, 0)	1938	0.5302
Alpha-Beta đã giảm được 68.15% số trạng thái cần xét!			
=====			
TEST CASE: 3. AI tấn công để thắng (Độ sâu: 2)			
=====			
Thuật toán	Nước đi	Số node đã xét	Thời gian (s)

Minimax	(2, 4)	5931	1.6729
Alpha-Beta	(2, 4)	837	0.2286
Alpha-Beta đã giảm được 85.89% số trạng thái cần xét!			

- Tại Độ sâu (Depth) = 3 (Bùng nổ tổ hợp):

- Khi tăng thêm 1 độ sâu, số lượng trạng thái của Minimax tăng vọt theo cấp số nhân. Tại Test 2, Minimax phải duyệt tới 450,837 node (tốn gần 150 giây).
- Trong khi đó, Alpha-Beta chứng minh sức mạnh cắt nhánh triệt để khi chỉ duyệt 62,490 node (tốn ~19.4 giây), giảm thiểu tới 86.14%.
- Đặc biệt tại Test 3 (AI có cơ hội thắng ngay), Alpha-Beta tìm thấy nhánh thắng từ sớm và cắt bỏ toàn bộ các nhánh vô ích còn lại. Số node duyệt giảm từ 450,683 xuống chỉ còn 7,691 node, tiết kiệm được 98.29% tài nguyên tính toán.

```

TEST CASE: 2. AI cần chặn người chơi (Độ sâu: 3)
=====
Thuật toán      | Nước đi   | Số node đã xét | Thời gian (s)
-----
Minimax         | (0, 0)    | 450837         | 149.7246
Alpha-Beta      | (0, 0)    | 62490          | 19.4402
Alpha-Beta đã giảm được 86.14% số trạng thái cần xét!

=====
TEST CASE: 3. AI tấn công để thắng (Độ sâu: 3)
=====
Thuật toán      | Nước đi   | Số node đã xét | Thời gian (s)
-----
Minimax         | (0, 0)    | 450683         | 137.4403
Alpha-Beta      | (0, 0)    | 7691           | 2.1647
Alpha-Beta đã giảm được 98.29% số trạng thái cần xét!

```

5. Ưu điểm, Hạn chế và Hướng cải tiến

- Ưu điểm:
 - Thuật toán xử lý chuẩn xác luật chơi Caro.
 - Hàm đánh giá nhận diện tốt các tình huống nguy hiểm, giúp AI biết phòng thủ kịp thời thay vì chỉ lo tấn công.
 - Alpha-Beta được cài đặt chuẩn, giúp tăng độ sâu tìm kiếm lên 3-4 mà chương trình vẫn phản hồi trong thời gian chấp nhận được.
- Hạn chế:
 - Hàm sinh nước đi `get_valid_moves()` hiện đang duyệt toàn bộ bàn cờ và trả về danh sách ô trống theo thứ tự tĩnh (từ trên xuống dưới, trái sang phải).
 - Thuật toán đôi khi xét những ô trống ở quá xa vùng đang giao tranh, gây lãng phí tài nguyên.
- Hướng cải tiến:

- Move Ordering (Sắp xếp nước đi): Sắp xếp danh sách các ô trống ưu tiên những ô nằm gần các quân cờ đã đánh. Điều này giúp Alpha-Beta tìm ra nhánh "tốt" sớm hơn, từ đó tăng tỷ lệ cắt nhánh lên mức tối đa.
- Zobrist Hashing: Áp dụng bảng băm để lưu lại điểm số của các trạng thái đã xét, tránh việc AI phải tính toán lại một thế cờ nếu nó được sinh ra từ các thứ tự đi cờ khác nhau.